

STUDENT INTERNSHIP PROJECT · AI & CYBERSECURITY

AI Linux Security Assistant

An intelligent tutor that explains Linux commands, analyzes file permissions, and recommends secure hardening practices — without ever executing user input.

Basaaria Imroz

B.E. CSE (AI & ML)

AI & Cybersecurity Internship

MRIIRS In collaboration with Ditto Security

What We'll Cover

01

Problem Statement

Why beginners struggle with Linux command & permission risk

02

Existing vs. Proposed

Manual guesswork vs. a structured AI-assisted workflow

03

System Architecture

How the modules connect using a ReAct-style pipeline

04

Live Feature Walkthrough

Command explainer, permission analyzer, hardening & reports

05

Testing & Results

Verified test cases across command and permission scenarios

06

Scope, Limitations & Future Work

What's next for the assistant

THE PROBLEM

One Risky Command Is All It Takes

Students learning Linux often copy commands from search results or tutorials and run them without understanding the consequences. A command that looks like it “fixes” an access error can silently expose files, delete data, or grant every local user full control.

- ! No unified explanation of purpose, syntax & risk
- ! Permission strings (rwxrwxrwx) are manually decoded
- ! No immediate Low / Medium / High / Critical rating
- ! Hardening guidance is scattered across many guides

A COMMON MISTAKE

```
$ chmod 777 script.sh
```

What it actually does: grants read, write, and execute access to the owner, the group, AND every other user on the system.

RISK LEVEL

CRITICAL

Safer alternative: `chmod 750 (script/directory)` or `chmod 640 (private file)`

Existing Workflow vs. Proposed Assistant

Existing Approach

- Search engines, tutorials & trial-and-error
- Manual decoding of permission triplets
- Risk awareness depends on learner experience
- Copy-pasted commands run without review
- Manual notes / screenshots for documentation

AI Linux Security Assistant

- One dashboard: command, output & permission input
- Automatic owner / group / other breakdown
- Deterministic Low–Critical risk classification
- Commands are parsed only — never executed
- One-click downloadable PDF security report

What the Project Set Out to Do

1

Explain

Translate raw Linux commands into beginner-friendly purpose, syntax & usage.

2

Analyze

Break down symbolic & octal permissions into owner / group / other access.

3

Classify

Rate every finding Low, Medium, High, or Critical using consistent rules.

4

Recommend

Suggest least-privilege alternatives and safe security commands.

5

Harden

Provide a reusable checklist covering SSH, firewall, accounts & updates.

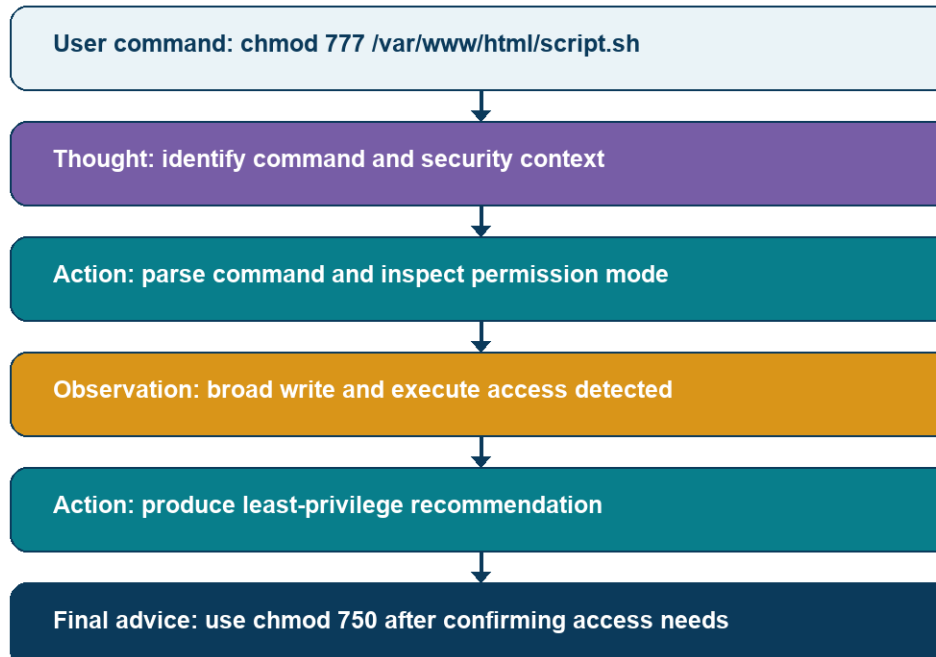
6

Report

Generate a downloadable PDF record of every assessment for review.

Modular Architecture & ReAct Workflow

ReAct-Inspired Security Analysis Workflow



REACT REASONING LOOP

- 1 Thought** Identify whether input is a command, output, or permission string.
- 2 Action** Call the matching tool: Command Parser or Permission Analyzer.
- 3 Observation** Collect structured findings and an initial risk rating.
- 4 Action** Run Hardening & Security Command Suggestion tools.
- 5 Result** Agent aggregates a security score and renders the report.

Built with a Lightweight, Free & Open-Source Stack

Python 3.11+

Core language for parsing, rules & orchestration

Streamlit

Interactive local/web dashboard, zero front-end code

ReportLab

Generates the downloadable PDF security report

Pandas / Plotly

Structured display & the security-score gauge

Rule-based Engine

Offline command & permission knowledge base

Optional LLM Hook

OpenAI / Gemini / Ollama for richer summaries

Runs fully offline — no API key or paid service required for the baseline rule-based experience.

Four Tools Working Together

01

Command Explanation Tool

Purpose, syntax, options, typical usage & security considerations for common commands (chmod, sudo, ssh, ufw, iptables & more).

02

Permission Analyzer

Detects world-writable/executable files, weak modes, SUID/SGID bits and converts symbolic ↔ octal notation.

03

Hardening Recommendation Tool

A curated checklist: SSH config, firewall policy, account hygiene, Fail2Ban, patching & audit logging.

04

Security Report Generator

Combines every finding into a structured, downloadable PDF with a final security score out of 100.

Enter a Command or Permission String

The sidebar keeps input minimal: a Linux command field and a permission-string field. No file uploads, no server setup.

Deploy ⋮

Analysis input

Linux command

chmod 777 /var/www/html/script.sh

Permission string

-rwxrwxrwx

Terminal output (optional)

Paste an ls -l line here

Analyze security

Commands are parsed locally; the app never executes them.

 **AI Linux Security Assistant**

Understand commands, audit permissions, and learn secure Linux administration—without executing anything on your system.

Enter a command or permission string in the sidebar, then select **Analyze security**. Example: `chmod 777 file.txt`.

Instant Risk Score & Security Findings

For `chmod 777 /var/www/html/script.sh` with `-rwxrwxrwx`: a 5/100 score, Critical command & permission risk, plain-language findings, and the transparent Thought → Action → Observation trace.

Analysis input

Linux command

chmod 777 /var/www/html/script.sh

Permission string

-rwxrwxrwx 1 user user 123 Jul 22 script.

Terminal output (optional)

Paste an ls -l line here

Analyze security

Commands are parsed locally; the app never executes them.

Deploy

Security overview

Command explainer

Permission analyzer

Hardening guide

Security score

5/100

Command risk

Critical

Permission risk

Critical

Security posture



Transparent analysis workflow

Thought — Identify the command and its security context.

Action — Run command explanation tool (local parser).

Observation — chmod risk: Critical

Action — Run permission analyzer (local parser).

Observation — Permission risk: Critical

Action — Generate least-privilege hardening recommendations.

One-Click Downloadable PDF Report

AI Linux Security Assistant Report

Generated: 2026-07-22 23:03

Security score: 5/100

Command	chmod
Purpose	Changes file or directory access permissions.
Command risk	Critical
Permissions	-rwxrwxrwx 1 user user 123 Jul 22 script.sh
Permission risk	Critical

Security findings

- Use the least-permissive mode needed. Avoid world-writable (777/666) permissions.
- The supplied mode grants broad write or execute access, violating least privilege.
- World-writable: any local user can modify this item.
- World-executable: every user can execute or traverse this item.
- Group members can write; confirm this group is intentionally trusted.

Recommended commands

```
chmod 750 # or chmod 640
```

Hardening checklist

Protect SSH: Disable direct root login, use SSH keys, and limit allowed users.
Edit /etc/ssh/sshd_config: PermitRootLogin no; PasswordAuthentication no. Test a second session before reloading SSH.

Enable a firewall: Allow only services that the system actually provides.
sudo ufw allow OpenSSH && sudo ufw enable # Ubuntu/Debian with UFW

Apply updates: Security updates close known vulnerabilities.
sudo apt update && sudo apt upgrade # review package changes first

Remove unused services: Fewer running services means a smaller attack surface.
systemctl --type=service --state=running # review before disabling anything

Use least privilege: Keep files private by default and avoid shared write access.
find /path -xdev -type f -perm -0002 -ls # audit world-writable files

Strengthen authentication: Use unique passwords, MFA where available, and account lockout policy appropriate to your environment.
passwd # change the current account password

Monitor logs: Review authentication and service logs for unexpected activity.
sudo journalctl -p warning..alert --since today

- ✓ Security score, command, and permission summary at a glance
- ✓ Findings written in plain, beginner-friendly language
- ✓ Recommended commands for the safer alternative
- ✓ Full hardening checklist with example commands
- ✓ Ready to submit as classroom / academic evidence

Verified Across Representative Test Scenarios

7 / 7

TEST CASES PASSED

Covering chmod risk, world-writable permissions, safe low-risk modes, recursive deletion, remote-script execution, SSH usage & malformed input handling.

ID	Scenario	Expected Result	Status
TC01	chmod 777 file.txt	Critical risk + safer 750/640 suggestion	Pass
TC02	Permission string -rwxrwxrwx	World-writable & world-executable findings	Pass
TC03	chmod 640 file.txt	Low risk, numeric mode 640 confirmed	Pass
TC04	rm -rf /var/tmp/testdir	High/Critical warning on recursive delete	Pass
TC05	curl http://x sh	Critical warning on piping remote code	Pass
TC06	ssh user@server	Purpose explained + key-based auth advice	Pass
TC07	Malformed permission input	Friendly format guidance, no crash	Pass

Scope, Limitations & Future Enhancements

In Scope Today

- chmod, chown, sudo, rm, ssh, ufw, systemctl & more
- Symbolic & octal permission analysis
- Hardening checklist + PDF report export

Current Limitations

- No live scan of a real Linux host
- No ACL / SELinux policy parsing
- Score is educational, not an enterprise rating

Future Enhancements

- Optional consent-based LLM for richer answers
- Distribution-aware advice (Ubuntu/RHEL/Kali)
- Docker-based safe lab mode for demos

CONCLUSION

Pause. Understand. Then Act.

The AI Linux Security Assistant bridges Linux command learning and cybersecurity awareness — encouraging every learner to understand a command, inspect its permissions, weigh the risk, and choose the least-privilege option before touching a real system.

Thank You

Basaaria Imroz · AI Linux Security Assistant